

CIS 5800, Machine Perception, Spring 2026
Homework 1 (Coding)
Due: Thursday Feb. 19 2026, 11:59pm ET
Version of 2026/02/06, 03:36GMT

Instructions

- This is the coding part for Homework 1 and worth **40 points**.
- You need to check the released **Jupyter Notebook** and fill in all the sections where ##### STUDENT CODE START ##### is indicated. You can work on it either using [Google Colab](#) or running it locally on your computer.
- Start early! Please post your questions on [Ed Discussion](#) (using the appropriate thread category) or come to office hours!

Submission

- For this coding part, you need to submit **3 files (.ipynb, .py and .zip)** in total to the [Gradescope](#). Please check the instruction in the notebook for more details.
- To convert your **.ipynb** to **.py** code (for autograder), you can either use "Download" in Google Colab or use other tools to export (e.g. VSCode can export **.py** from **.ipynb** with **Jupyter Extension**).

1 Introduction

In this programming assignment, we will cover basic Euclidean transformations(i.e. Scaling, Translation, and Rotation) as well as Projective Transformations(i.e. Homographies). We note that for all programming implementations, any OpenCV functions are strictly prohibited.

2 Technical Details

You will be provided with a Jupyter Notebook that highlights the sections where you must complete the code. Your task is to implement the geometric transformations described below, along with a homography.

3 Geometric Transformations (20 pts)

This section focuses on the implementation of fundamental geometric transformations in two-dimensional space. Specifically, we consider scaling, translation, and rotation, which are commonly used operations in computer vision and image processing. Each transformation will be implemented **manually** (i.e. from scratch), without relying on any built-in OpenCV functions. The goal of this section is to develop a clear understanding of how these transformations affect image coordinates and how they can be applied to manipulate images programmatically.

3.1 Scaling

For this portion, we focus on image scaling. Specifically, we aim to upsample (i.e., increase the size of) the input image. Using the provided image (`messi.jpg`), upscale the image by factors of 2, 3, and 4. Plot the original image together with the $2\times$, $3\times$, and $4\times$ scaled images in a 1×4 layout, displayed side by side. The final result should resemble the following.



Figure 1: Scaled Images

3.2 Translation

In this portion, we focus on image translation. Using the given input image (`messi.jpg`), translate the image by different amounts in the vertical and horizontal directions. Specifically, apply translations for each $(\text{translation_y}, \text{translation_x}) \in \{(10, 20), (20, 30), (30, 40)\}$. Plot the original image along with the translated images in a 1×4 side-by-side layout. The final result should resemble the example shown below.

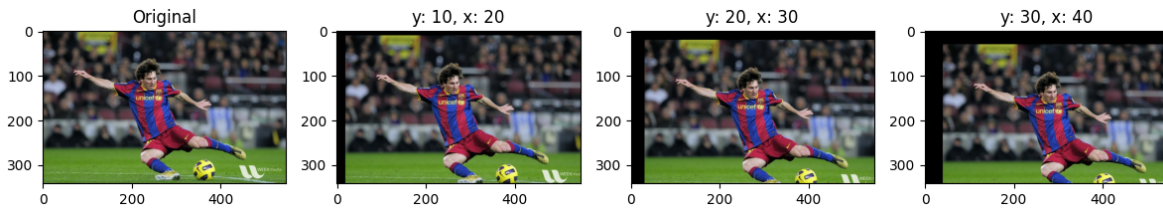


Figure 2: Translated Images

3.3 Rotation

Next, we focus on image rotation. Using the same input image, rotate the image counterclockwise by 45° , 90° , and 135° . Plot the original image together with the rotated images in a 1×4 layout, displayed side by side. The final result should resemble the example shown below.

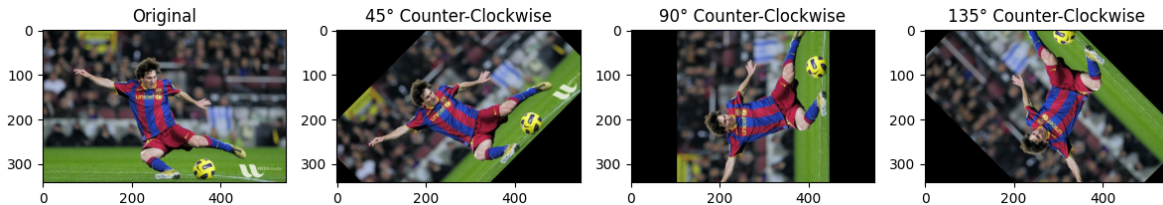


Figure 3: Rotated Images

4 Homography-Based Image Projection (20 pts)

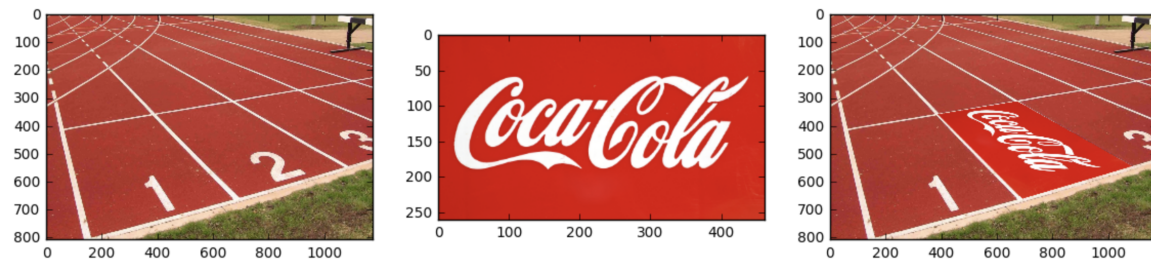


Figure 4: Homography Projection of Coca-Cola Logo onto Track

In this section, you will use a homography to project the Coca-Cola logo onto a planar region of a target image. Given an image of a running track and a separate image containing the Coca-Cola logo, your goal is to warp the logo so that it appears naturally painted onto the surface of the track, as shown in the example above. Since both the logo and the target region on the track can be approximated as planar surfaces, their

geometric relationship can be modeled using a single homography. You will estimate this homography from corresponding corner points and use it to map pixels from the logo image onto the specified region of the track image.

4.1 Homography Estimation

To project one image patch onto another, we need, for each point inside the track, to find the corresponding point from the logo image to copy over. In other words, we need to calculate the homography between the two image patches. This homography is a 3x3 matrix that satisfies the following:

$$\mathbf{x}_{logo} \sim H\mathbf{x}_{track} \quad (1)$$

Or, equivalently:

$$\lambda\mathbf{x}_{logo} = H\mathbf{x}_{track} \quad (2)$$

Where $\mathbf{x}_{logo}$ and $\mathbf{x}_{track}$ are homogeneous image coordinates from each patch and λ is some scaling constant. To calculate the homography needed for this projection, we provide, for each image, the corners of the patches that we would like you to warp between in each image. You must calculate the homography using the provided corner points and the technique covered in the lectures and Appendix A. You can then warp each image point using H to find its corresponding point in the logo (note that the homography equation is estimated up to a scalar, so you will need to divide $H\mathbf{x}_{image}$ by the third term, which is λ), and then return the set of corresponding points as a matrix.

4.2 Inverse Warping

You may be wondering why we are calculating the projection from the track image to the logo image, when we want to project the logo image onto track. We do this because, if we compute the inverse homography, and project all the logo points onto the track, we will most likely have the case where multiple logo points project to one track pixel (due to rounding of the pixels), while other pixels may have no logo points at all. This results in 'holes' in the track where no logo points are mapped. To avoid this, we calculate the projection from track points to logo points to guarantee that every track point gets a point from the logo.

We can then replace every point in the track ($\mathbf{x}_{track}$) with the corresponding point in the logo ($\mathbf{x}_{logo}$) using the correspondences ($\mathbf{x}_{track}, \mathbf{x}_{logo}$). This is already done for you in **inverse_warping**, and you should not need to change it.

Appendix A: Calculating Homographies

As we learned in the lectures, a homography H maps a set of points $\mathbf{x} = \begin{pmatrix} x & y & 1 \end{pmatrix}^T$ to another set of points $\mathbf{x}' = \begin{pmatrix} x' & y' & 1 \end{pmatrix}$ up to a scalar:

$$\mathbf{x}' \sim H\mathbf{x} \quad (3)$$

$$\lambda \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (4)$$

$$\lambda x' = h_{11}x + h_{12}y + h_{13} \quad (5)$$

$$\lambda y' = h_{21}x + h_{22}y + h_{23} \quad (6)$$

$$\lambda = h_{31}x + h_{32}y + h_{33} \quad (7)$$

In order to recover x' and y' , we can divide equations (5) and (6) by (7):

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}} \quad (8)$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}} \quad (9)$$

Rearranging the terms above, we can get a set of equations that is linear in the terms of H :

$$-h_{11}x - h_{12}y - h_{13} + h_{31}xx' + h_{32}yx' + h_{33}x' = 0 \quad (10)$$

$$-h_{21}x - h_{22}y - h_{23} + h_{31}xy' + h_{32}yy' + h_{33}y' = 0 \quad (11)$$

Finally, we can write the above as a matrix equation:

$$\begin{pmatrix} a_x \\ a_y \end{pmatrix} h = 0 \quad (12)$$

Where:

$$a_x = \begin{pmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \end{pmatrix} \quad (13)$$

$$a_y = \begin{pmatrix} 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{pmatrix} \quad (14)$$

$$h = \begin{pmatrix} h_{11} & h_{12} & h_{13} & h_{21} & h_{22} & h_{23} & h_{31} & h_{32} & h_{33} \end{pmatrix}^T \quad (15)$$

Our matrix H has 8 degrees of freedom, and so, as each point gives 2 sets of equations, we will need 4 points to solve for h uniquely. So, given four points (such as the

corners provided for this assignment), we can generate vectors a_x and a_y for each, and concatenate them together:

$$A = \begin{pmatrix} a_{x,1} \\ a_{y,1} \\ \vdots \\ a_{x,n} \\ a_{y,n} \end{pmatrix} \quad (16)$$

Our problem is now:

$$Ah = 0 \quad (17)$$

As A is a 8x9 matrix, there is a unique null space. Normally, we can use MATLAB's **null** function, however, due to noise in our measurements, there may not be an h such that Ah is exactly 0. Instead, we have, for some small $\vec{\epsilon}$:

$$Ah = \vec{\epsilon} \quad (18)$$

To resolve this issue, we can find the vector h that minimizes the norm of this $\vec{\epsilon}$. To do this, we must use the SVD, which we will cover in week 3. For this project, all you need to know is that you need to run the command:

$$[U, S, V] = \text{svd}(A);$$

The vector h will then be the last column of V , and you can then construct the 3x3 homography matrix by reshaping the 9x1 h vector.